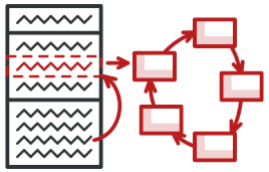
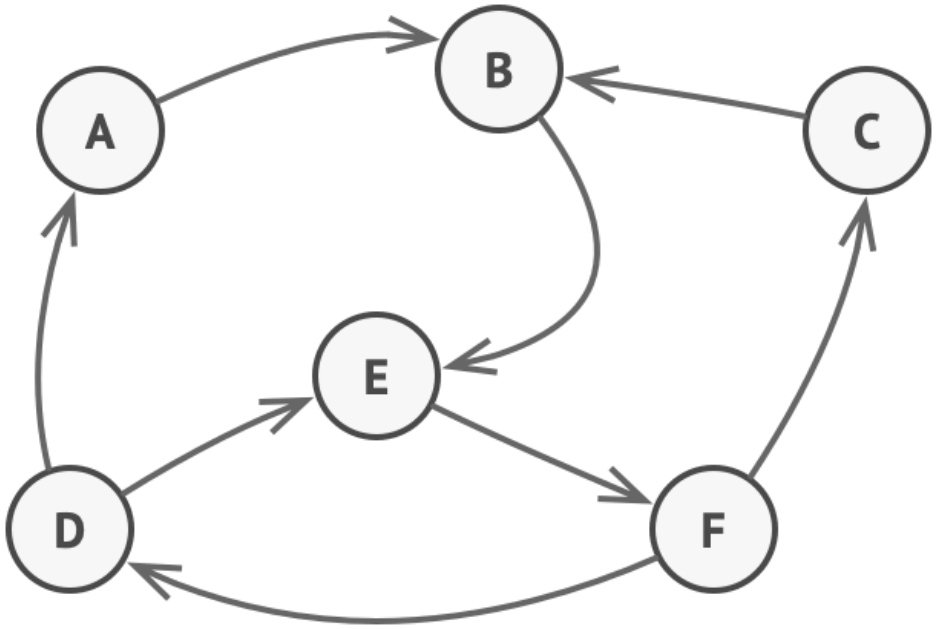
Поведенческие паттерны. Состояние



Паттерн Состояние
State (Конечный автомат)

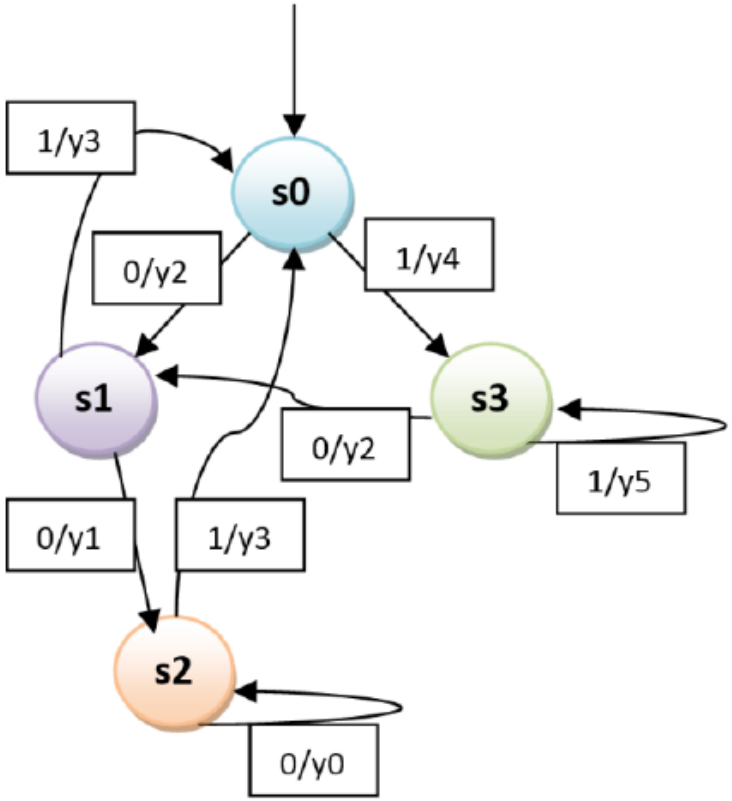


Состояние — это поведенческий паттерн, позволяющий динамически *изменять поведение* объекта при смене его *состояния*.

Поведения, зависящие от состояния, реализуются как отдельные классы. Первоначальный класс хранит ссылку на один из таких объектов-состояний и делегирует ему работу.

Проблема

Конечный автомат - модель дискретного устройства, имеющего один вход, один выход и в каждый момент времени находящегося в одном состоянии из множества возможных.



Программа может находиться в одном из нескольких состояний, которые сменяют друг друга.

Набор этих состояний, а также переходов между ними, предопределён и *конечен*.

Находясь в разных состояниях, программа может по-разному реагировать на одни и те же события, которые происходят с ней.

Проблема

Возможные состояния документа и переходы между ними.



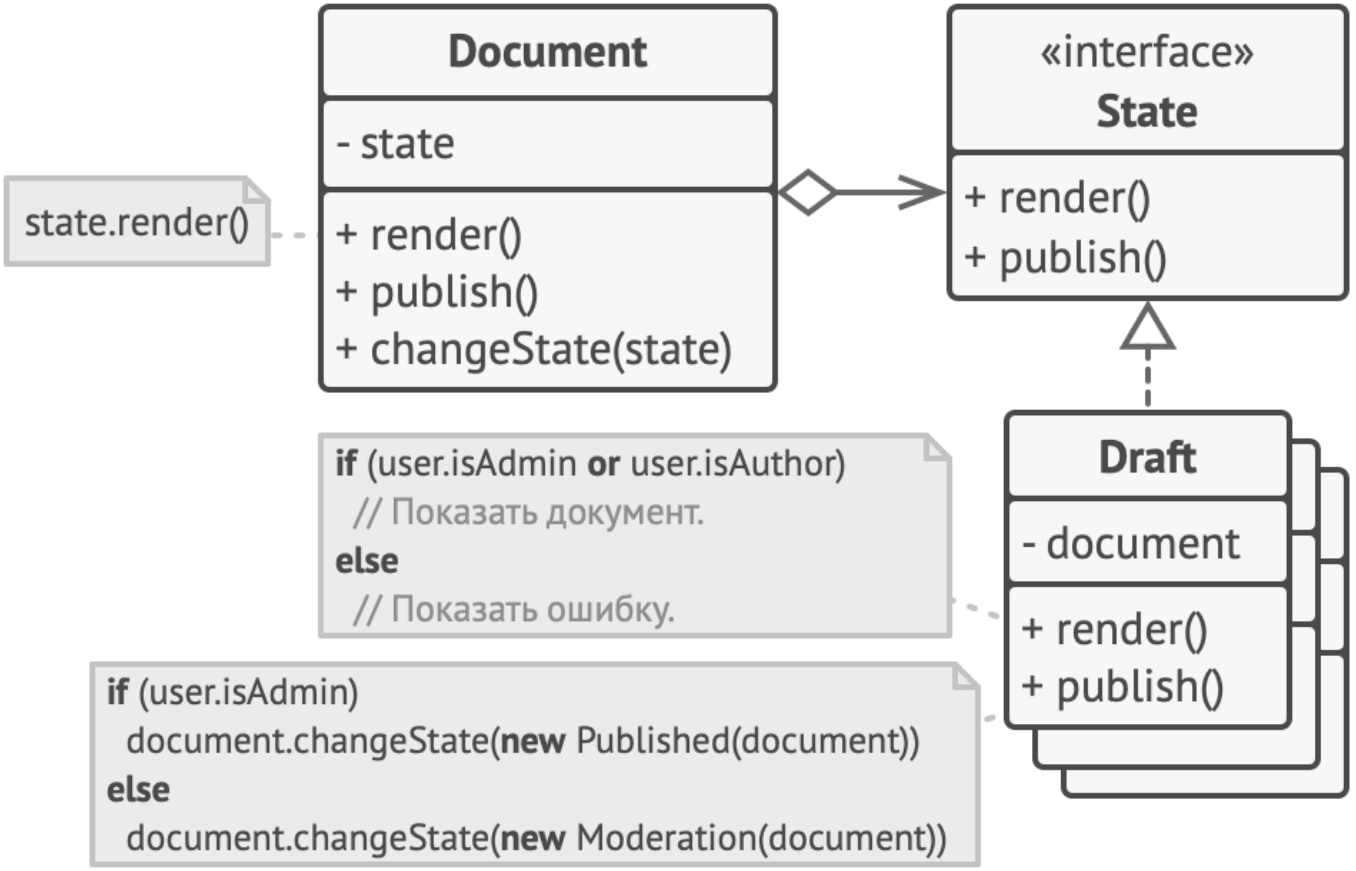
Реализация КА на if

```
class Document is
    field state: string
    // ...
    method publish() is
        switch (state)
            "draft":
                state = "moderation"
                break
            "moderation":
                if (currentUser.role == "admin")
                    state = "published"
                break
            "published":
                // Do nothing.
                break
    // ...
```

Проблема – если состояний **много**

Решение

Создать отдельные классы для каждого состояния, в котором может пребывать объект, и вынести туда поведения, соответствующие этим состояниям.



Первоначальный объект – *контекст* - содержит ссылку на один из объектов-состояний и делегирует ему работу, зависящую от состояния.

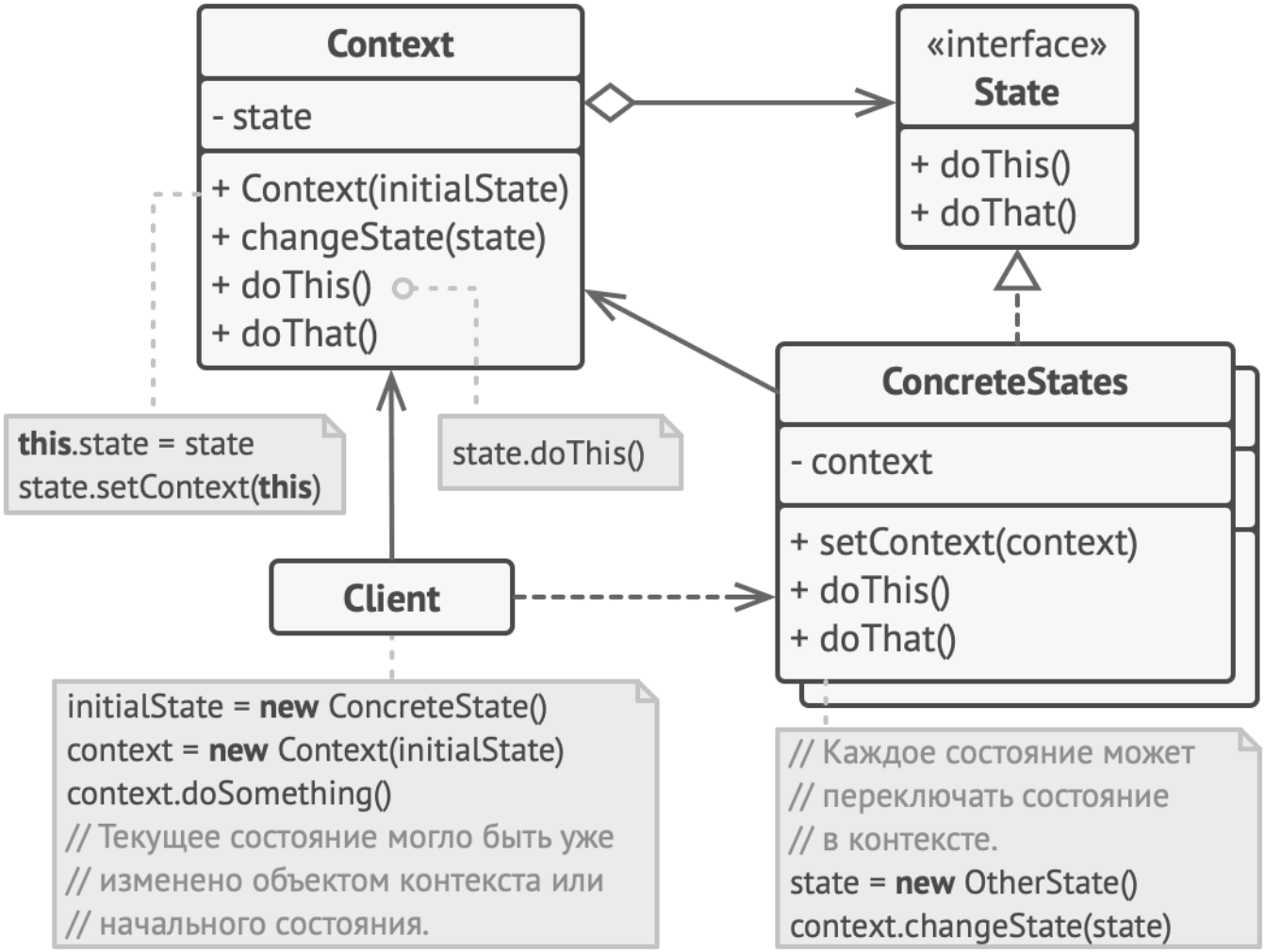
Структура паттерна

[1]**Контекст** хранит ссылку на объект состояния и делегирует ему часть работы, зависящей от состояний. Контекст работает с этим объектом через общий интерфейс состояний. Контекст должен иметь метод для присваивания ему нового объекта-состояния.

[2]**Состояние** описывает общий интерфейс для всех конкретных состояний.

[3]**Конкретные состояния** реализуют поведения, связанные с определённым состоянием контекста.

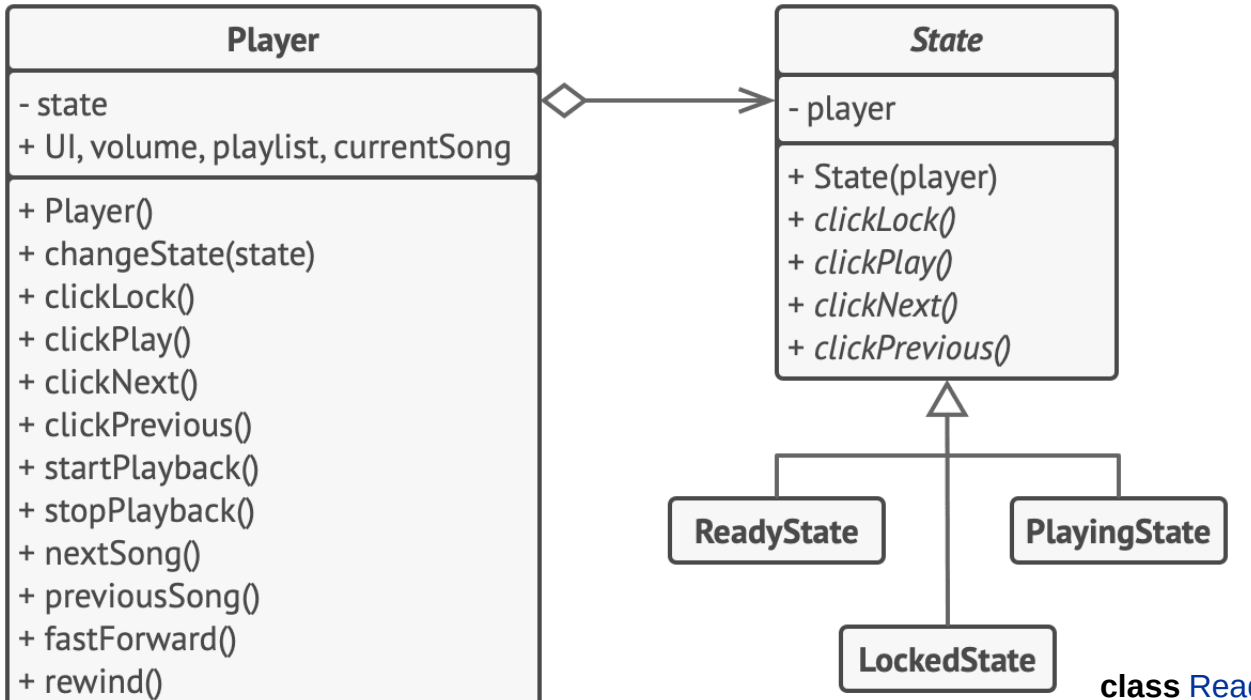
Состояние может иметь обратную ссылку на объект контекста. Через неё не только удобно получать из контекста нужную информацию, но и осуществлять смену его состояния.



[4]И контекст, и объекты конкретных состояний могут решать, когда и какое следующее состояние будет выбрано. Чтобы переключить состояние, нужно подать другой объект-состояние в контекст.

Пример

Изменение поведения проигрывателя с помощью состояний




```
abstract class State is

    abstract method clickLock()
    abstract method clickPlay()
    abstract method clickNext()
    abstract method clickPrevious()
```

```
class LockedState extends State is

    method clickPlay() is
        // Ничего не делать.

    method clickNext() is
        // Ничего не делать.

    method clickPrevious() is
        // Ничего не делать.
```

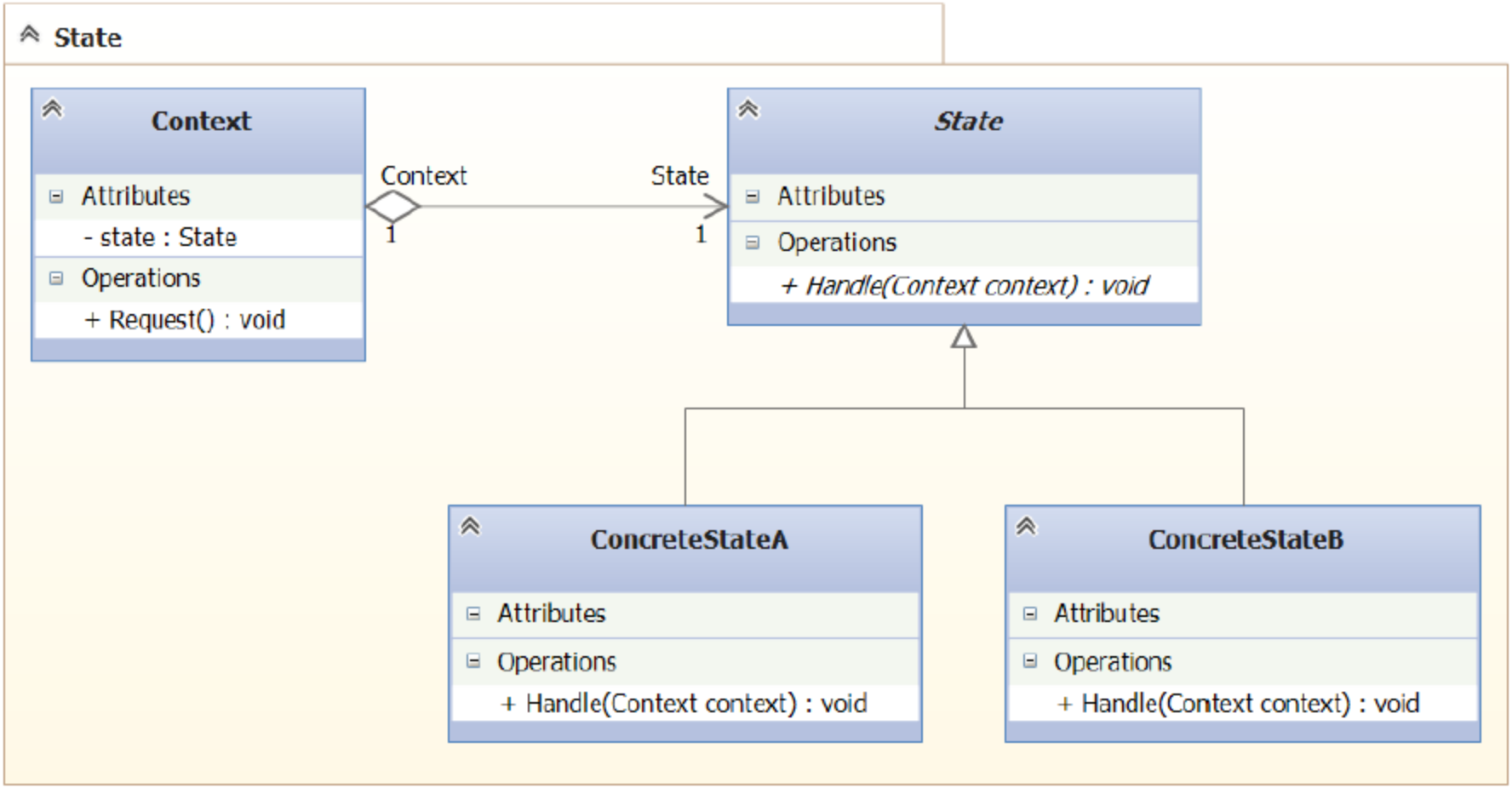
```
class ReadyState extends State is
    method clickLock() is
        player.changeState(new LockedState(player))

    method clickPlay() is
        player.startPlayback()
        player.changeState(new PlayingState(player))

    method clickNext() is
        player.nextSong()

    method clickPrevious() is
        player.previousSong()
```

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        Context context = new Context(new ConcreteStateA());
        context.Request();
        context.Request();
    }
}
```

`Context` - **Контекст**: объектно-ориентированное представление конечного автомата. Предоставляет интерфейс взаимодействия, посредством которого клиенты могут оказывать влияние на конечный автомат, например, посредством передачи входных сигналов. Объект класса `Context` содержит ссылку на экземпляр текущего состояния.

```
class Context
{
    public State State { get; set; }

    public Context(State state)
    {
        this.State = state;
    }

    public void Request()
    {
        this.State.Handle(this);
    }
}
```

`State` - **Состояние**:
Задает интерфейс взаимодействия с «объектом-состоянием».

```
abstract class State
{
    public abstract void Handle(Context context);
}
```

`ConcreteState` - **Конкретное состояние**: Реализует поведение (функции выхода и функции перехода) ассоциированное с определенным состоянием.

```
class ConcreteStateA : State
{
    public override void Handle(Context context)
    {
        context.State = new ConcreteStateB();
    }
}
```

```
class ConcreteStateB : State
{
    public override void Handle(Context context)
    {
        context.State = new ConcreteStateA();
    }
}
```

Отношения между участниками

Отношения между классами

- Конкретные классы (ConcreteStateA и ConcreteStateB) связанны связью отношения наследования с абстрактным классом State.
- Конкретный класс Context связан связью отношения агрегации с абстрактным классом State.
- Отношения между объектами
- Объект класса Context передает клиентские запросы объекту класса ConcreteState.
- Объект класса Context может передать себя в качестве аргумента объекту типа State, который будет обрабатывать запрос. Такой подход позволит объекту-состоянию получить доступ к контексту и произвести замену состояния.
- Объект класса Context обладает интерфейсом для конфигурирования его нужным состоянием. Один раз сконфигурировав контекст, клиенты должны отказаться от дальнейшей конфигурации. Чаще всего объекты ConcreteState сами принимают решение о том, при каких условиях и в каком порядке происходит изменение состояния.

Применение

1. Когда есть объект, поведение которого кардинально меняется в зависимости от внутреннего состояния, причём типов состояний много, и их код подвержен изменениям.

Паттерн предлагает выделить в собственные классы все поля и методы, связанные с определёнными состояниями. Первоначальный объект ссылается на один из объектов-состояний, делегируя ему часть своей работы. Для изменения состояния в контекст достаточно будет подставить другой объект-состояние.

2. Когда код класса содержит множество сложных и сходных по структуре, условных операторов, которые выбирают поведения в зависимости от текущих значений полей класса.

Паттерн предлагает переместить каждую ветку такого условного оператора в собственный класс. Тут же можно поместить и все поля, связанные с данным состоянием.

3. Когда используется табличная машина состояний, построенная на условных операторах, с дублированием кода для похожих состояний и переходов.

Паттерн *Состояние* позволяет реализовать иерархическую машину состояний, базирующуюся на наследовании. Похожие состояния можно наследовать от одного родительского класса и вынести туда весь дублирующий код.

Шаги реализации

1. **Определить с класс, который будет играть роль контекста**. Это может быть как существующий класс, в котором уже есть зависимость от состояния, так и новый класс.

2. **Создать общий интерфейс состояний**. Он должен описывать методы, общие для всех состояний, обнаруженных в контексте. *Не всё поведение контекста нужно переносить в состояние, а только то, которое зависит от состояний*.

3. **Для каждого фактического состояния создать класс, реализующий интерфейс состояния.** Переместить код, связанный с конкретными состояниями в нужные классы. Все методы интерфейса состояния должны быть реализованы во всех классах состояний.

Проблема: поведение может зависеть от приватных полей или методов контекста, к которым нет доступа из объекта состояния.

Решение: 1) оставить поведение внутри контекста, вызывая его из объекта состояния;

2) сделать классы состояний вложенными в класс контекста, в этом случае они получат доступ ко всем приватным частям контекста.

4. **Создать в контексте поле для хранения объектов-состояний,** а также публичный метод для изменения значения этого поля.

5. **Старые методы контекста, в которых находился зависимый от состояния код**, заменить на вызовы соответствующих методов объекта-состояния.

6. **В зависимости от логики предметной области**, разместить код, который переключает состояние контекста либо внутри контекста, либо внутри классов конкретных состояний.

Преимущества и недостатки

+ Избавляет от множества условных операторов машины состояний, перегруженных кодом.

+ Концентрирует в одном месте код, связанный с определённым состоянием.

+ Упрощает код контекста.

- Может неоправданно усложнить код, если состояний мало и они редко меняются.